Website navigation

Side bar

The sidebar is the main navigation for documentation-style sites. Configure it with `[sidebar]`; a section can list pages one by one:

```toml
[sidebar]

[[sidebar.section]]
title = "Guide"

  [[sidebar.section.item]]
  target = "install.typ"
```

or include several pages with a glob:

```toml
[[sidebar.section.item]]
glob = "guide/*.typ"
```

Use `target` for one source page and `glob` for a list of source pages. Page targets point to Typst source files, not rendered `.html` files. *Calepin* resolves those pages and writes the right `.html` links in the generated site.

Sidebar items must be nested under `[[sidebar.section]]`, but the section does not need a title. Use an untitled section when you want items to appear as a plain unheaded list:

```toml
[sidebar]

[[sidebar.section]]
  [[sidebar.section.item]]
  target = "index.typ"

  [[sidebar.section.item]]
  target = "getting-started.typ"
```

The sidebar label comes from the page source, not from `calepin.toml`. Put the label in the page's website metadata:

```typst
#set document(title: [Install])
#metadata((title: "Install")) <website-metadata>

#title()
```

If a page has no `website-metadata.title`, *Calepin* falls back to the document title, then the filename stem. This keeps multilingual sidebars in one place: each translated page carries its own translated title.

Use an external URL as `target` when you want a sidebar link to leave the site. External targets must set `label` because there is no page metadata to read:

```toml
[[sidebar.section.item]]
target = "https://example.com/reference"
label = "External reference"
```

Add non-link subheadings inside a section with an item that sets only `label`:

```toml
[[sidebar.section.item]]
label = "Language"

[[sidebar.section.item]]
target = "reference/syntax.typ"

[[sidebar.section.item]]
target = "reference/styling.typ"

[[sidebar.section.item]]
label = "Library"

[[sidebar.section.item]]
target = "reference/model.typ"
```

Subheadings are rendered in sidebar order with the `calepin-website-sidebar-subheading` class so themes can style them separately. They do not link anywhere, add build pages, or affect which folded section opens. A sidebar item with a `.typ` `target` or `glob` cannot also set `label`; page labels still come from page metadata.

If you do not configure a sidebar, *Calepin* builds one from `.typ` files in the source directory. Hidden files are skipped.

Titled sections are foldable: each page loads with the section that contains it open and the others folded. Opening a different section folds the previous one. To keep every section expanded instead, disable folding:

```toml
[sidebar]
fold = false
```

Table of contents

Pages can show an "On this page" table of contents built from their own headings (levels 1-3 by default). The `calepin` theme shows one by default; other themes, including `academic`, are opt-in.

Set a site-wide default with `[toc]`:

```toml
[toc]
enabled = true
depth = 2
floating = true
```

`depth` is the maximum heading level included, from 1 to 6. The table of contents floats with the viewport by default. Set `floating = false` to keep it in the normal page flow.

Override any field for a single page with `<website-metadata>`:

```typst
#metadata((toc: (enabled: false))) <website-metadata>
```

```typst
#metadata((toc: (depth: 2))) <website-metadata>
```

```typst
#metadata((toc: (floating: false))) <website-metadata>
```

Page metadata and `calepin.toml` merge field by field, so a page can override any TOC setting while inheriting the others. Unset `enabled` and `depth` values fall back to the theme defaults; `floating` defaults to `true`.

Site menus

Use `[menus]` for named navigation groups. Menu names describe what the links mean; themes decide where to render them. The bundled themes understand `main` and `social`. Custom themes can use any additional menu name.

```toml
[[menus.main]]
target = "index.typ"
label = "Home"
weight = 10

[[menus.social]]
target = "https://github.com/user/repo"
label = "{icon:github}"
aria-label = "GitHub"
```

Menu items use `target` or `glob`. Use a `.typ` `target` or `glob` for internal source pages; use any other `target` for external links or a literal already-rendered URL. Omit `label` for internal pages to use the page metadata title, document title, or filename stem.

Footer

Configure the site footer with `[[footer.item]]`. Footer items can be links or plain text rows for copyright and legal notices:

```toml
[[footer.item]]
label = "© 2026 Example"

[[footer.item]]
target = "https://example.com/privacy"
label = "Privacy"
```

A footer row with only `label` is rendered as text (no hyperlink).

Use `weight` to control ordering within one menu or footer. Lower weights appear first. Items without weights keep their config order after weighted items.

Labels can include Iconify icons with `{icon:...}`. If the prefix is omitted, *Calepin* uses `lucide`, so `{icon:github}` means `{icon:lucide:github}`. Icon prefixes are Iconify collection names. Search available icons in the Iconify icon sets browser.

For icon-only visible labels, set `aria-label` so screen readers get a human-readable name:

```toml
aria-label = "GitHub"
```

(If you omit `aria-label`, Calepin uses fallback text if available; for an icon-only label with no fallback text this will be less readable.)

Local SVG icons are also supported with source-relative paths:

```toml
[[menus.social]]
target = "https://example.com/project"
label = "{icon:assets/icons/project.svg} Project"
```

Local icon paths must stay inside the website source directory. *Calepin* sanitizes local and downloaded SVGs before inlining them.